



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №6
з дисципліни
“Технології розроблення програмного забезпечення”
на тему “Редактор зображень”

Виконав
студент групи ІА–22:
Гордун Д.В.

Перевірив
Мягкий М.Ю.

Київ 2024

Хід роботи

Додав код за посиланням:

<https://github.com/DmytroH25/trpz/pull/5/commits/c47c0631af424d9e6c447679e22852057270b312>

Теоретичні відомості

Принципи проектування SOLID Принципи проектування SOLID були введені Робертом Мартіном в його статті "Об'єктноорієнтоване проектування" і відносяться до розробки класів систем. Ось вони:

S (SRP) Принцип єдиного обов'язку На кожен об'єкт має бути покладений один єдиний обов'язок.

O (OCP) Принцип відкритості/закритості Програмні сутності мають бути відкриті для розширення, але закриті для зміни.

L (LSP) Принцип підстановки Барбари Лісков Об'єкти в програмі можуть бути замінені їх спадкоємцями без зміни властивостей програми.

I (ISP) Принцип розділення інтерфейсу Багато спеціалізованих інтерфейсів краще, ніж один універсальний.

D (DIP) Принцип інверсії залежностей Залежності усередині системи будуються на основі абстракцій. Модулі верхнього рівня не залежать від модулів нижнього рівня. Абстракції не повинні залежати від деталей. Деталі повинні залежати від абстракцій.

Принцип єдиної відповідальності (single responsibility principle) свідчить про необхідність створення класів, що відповідають не більше ніж за одну річ. Проблема, яка виникає коли один клас містить безліч обов'язків, - раніше всього, погана читаність коду (як правило, такі класи розростаються до неймовірних розмірів і стає практично неможливою їх подальша підтримка), і складність зміни. Наприклад, якщо служба звітів тісно інтегрована із службою друку (у одному класі), то зміни в одній із служб напевно приведуть до поломки інших. Цей принцип також веде до зменшення зв'язаності компоненту (low coupling) і підвищення цілісності елементів системи (high cohesion).

Принцип відкритості/закритості проголошує необхідність додавання можливості розширення без зміни початкових кодів самого компонента. Таким чином існує можливість зміни поведінки компонента без зачіпання початкових кодів, що може знадобитися у разі тестування (заміна окремих компонент заглушками) або додавання функціональності. Однією з реалізацій принципів відкритості/закритості є реалізація інтерфейсів або

абстрактних класів для специфікації семантики необхідних дій (контракт послуг, що надаються), і надання можливостей до власної реалізації цих інтерфейсів (конкретна реалізація).

Принцип підстановки Барбари Лисков стверджує, що підкласи в ієрархії класів повинні зберігати функціональність, аналогічну батьківським класам. Підстановка підкласів повинна залишити програму функціонувати тим же чином (коректно). Проблема походить з наступних міркувань: припустимо, реалізується клас калькулятор; створюється дочірній клас "цифровий калькулятор", але в перевантаженні методу "сума" використовується множення. Синтаксично, таке використання дочірніх класів не несе помилок (компілюється і запускається). Семантично, результат роботи програми абсолютно поміняється при заміні батьківського класу підкласом (кожного разу при підсумовуванні ми будемо насправді множити). Принцип свідчить про необхідність зберігати семантику базового класу в уникнення безглузвих помилок в програмному забезпеченні.

Основною ідеєю принципу розділення інтерфейсів є розбиття загального "контракту" (чи набору послуг, що надаються) програми на безліч дрібних інтерфейсних модулів, які відповідають за одну конкретну роботу. Такий підхід нагадує принцип одного обов'язку, але відноситься до усієї системи в цілому, і спрощує читання, розуміння і модифікацію системи, оскільки користувачі інтерфейсу бачать лише той "розріз" системи, який їм в даний момент потрібний.

Принцип інверсії залежностей стверджує про необхідність усунення залежностей модулів верхнього рівня від модулів нижнього рівня, оскільки і ті і інші повинні посилалися на абстракції (інтерфейси). Абстракції, у свою чергу, не повинні залежати від деталей, а навпаки. Найпростіше зрозуміти цей принцип на конкретному прикладі: нехай існує деяка служба, що виробляє конкретні розрахунки (назвемо її Служба 1); існує служба, яка має посилання на неї, і використовує для реалізації власної внутрішньої механіки (назвемо її служба 2). Реалізація цього принципу виглядатиме таким чином: у Служби 1 буде винесений загальний інтерфейс, і цей інтерфейс використовуватиметься в Службі 2 (підставлятися в конструктор, наприклад). Таким чином, служба 1 залежить від абстракції (загальний інтерфейс), і не впливає на роботу Служби 2 (модулі верхнього рівня не залежать від модулів нижнього рівня). Абстракція у свою чергу не залежить від деталей, оскільки є простою специфікацією можливих дій. Слід зазначити, що цей принцип добре поєднується з попередніми, і дозволяє складати чистіші застосування. Існує безліч реалізацій цього принципу - впровадження залежностей, система розширень, локатор служб.

Патерн Memento:

Шаблон використовується для збереження і відновлення стану об'єктів без порушення інкапсуляції. Об'єкт "мементо" служить виключно для збереження змін над початковим об'єктом (Originator). Лише початковий об'єкт має можливість зберігати і отримувати стан об'єкту "мементо" для власних цілей, цей об'єкт є "порожнім" для кого-небудь ще. Об'єкт Caretaker використовується для передачі і зберігання мементо об'єктів в системі. Таким чином вдається досягти наступних цілей:

- Зберігання стану повністю відділяється від початкових об'єктів, що полегшує їх реалізацію;
- Передача об'єктів мементо лягає на плечі Caretaker об'єктів, що дозволяє гнучкіше управляти станами об'єктів і спростити дизайн класів початкових об'єктів;
- Збереження і відновлення стану реалізовані у вигляді двох простих методів і є закритими для кого-небудь ще окрім початкових об'єктів, таким чином не порушуючи інкапсуляцію.

Реалізація шаблону Memento в проєкті

У нашому проєкті шаблон **Memento** використовується для збереження та відновлення стану зображення при редагуванні, забезпечуючи можливість реалізації функцій "**Undo**" (скасування змін) і "**Redo**" (повторення скасованих змін). Це дозволяє користувачам повертатися до попередніх версій зображення або відновлювати скасовані зміни, зберігаючи при цьому всю історію змін.

Коли застосовується новий ефект до зображення, його поточний стан зберігається у вигляді об'єкта ImageMemento і додається до стеку історії змін. Якщо користувач викликає "**Undo**", зображення відновлюється з попереднього стану в стеку історії, а поточний стан переміщається в стек для "**Redo**".

Якщо викликається "**Redo**", зображення повертається до наступного стану зі стеку redoStack.

```

@RequiredArgsConstructor 7 usages
public class ImageMemento {

    private final byte[] imageData;

    public byte[] getImageData() { 2 usages
        return imageData.clone(); // Повертаємо копію, щоб зберегти незмінність
    }
}

```

Рис.1 Клас Memento

```

private void saveToHistory(byte[] imageData) { 3 usages
    history.push(new ImageMemento(imageData)); // Зберігаємо стан в історію
}
}

```

```

public Image saveImage(MultipartFile file) throws IOException { 1 usage
    Image image = new Image();
    image.setContentType(file.getContentType());
    image.setFilename(file.getOriginalFilename());
    image.setImageData(file.getBytes());

    currentImageData = file.getBytes();
    saveToHistory(currentImageData); // Зберігаємо стан
    redoStack.clear(); // Очищуємо стек Redo
    return imageRepository.save(image);
}

```

```

public Long applyEffect(Long imageId) throws IOException { 3 usages
    Image image = getImageById(imageId).orElseThrow(
        () -> new IllegalArgumentException("Image not found"));
    Image clonedImage = image.clone();
    BufferedImage bufferedImage = readImageFromFile(image);
    byte[] modifiedImage = currentState.applyEffect(bufferedImage).getImageData();

    saveToHistory(currentImageData); // Зберігаємо попередній стан
    redoStack.clear(); // Очищаємо стек Redo
    currentImageData = modifiedImage; // Оновлюємо поточний стан
    clonedImage.setImageData(currentImageData);
    image.setImageData(clonedImage.getImageData());
    return image.getId();
}

public void undo() { 1 usage
    if (!history.isEmpty()) {
        redoStack.push(new ImageMemento(currentImageData)); // Зберігаємо для Redo
        ImageMemento previousState = history.pop();
        currentImageData = previousState.getImageData(); // Відновлюємо попередній стан
    }
}

public void redo() { 1 usage
    if (!redoStack.isEmpty()) {
        saveToHistory(currentImageData); // Зберігаємо для Undo
        ImageMemento nextState = redoStack.pop();
        currentImageData = nextState.getImageData(); // Відновлюємо наступний стан
    }
}
}

```

Рис. 2 Застосування Memento

Висновок: На цій лабораторній роботі було досліджено та кілька шаблонів проектування, таких як «**Abstract Factory**», «**Factory Method**», «**Memento**», «**Observer**», «**Decorator**», а також розглянуто принципи проектування **SOLID**. Кожен із цих підходів зробив внесок у створення гнучкого, масштабованого та підтримуваного програмного забезпечення, але особливий акцент було зроблено на шаблоні **Memento** через його безпосереднє застосування в нашому проєкті. Шаблон **Memento** дозволив реалізувати функціонал скасування змін (**Undo**) та повторення скасованих дій (**Redo**), що є важливим аспектом для інтуїтивного користувацького досвіду.